

B 4.1.3 Construct and apply linked lists - Python (HL Only)



B 4.1.3 Construct and apply linked lists: singly, doubly and circular.

The syllabus subtopic B4.1.3 focuses on the practical skill of constructing and applying various types of linked lists within the Diploma Programme (DP) Computer Science course, specifically for Higher Level (HL) students

1. Singly Linked Lists (SLL)

A singly linked list is a linear data structure where elements are stored in nodes. Each node contains data and a reference (**next**) to the next node in the sequence. The list is accessed via a **head** reference that points to the first node.

The **Node** Class

The fundamental building block.

```
class Node:
    def __init__(self, data):
        self.data = data # The data value of the node
        self.next = None # A reference to the next node, initially None
```

The **SinglyLinkedList** Class

This class manages the collection of nodes and contains the methods to operate on the list.

```
class SinglyLinkedList:
    def __init__(self):
        self.head = None # The starting point of the list

    # ... methods go here
```

SLL Operations

Traversal / Display To visit every node, we start at the **head** and follow the **next** references until we reach **None**.

```
# Inside SinglyLinkedList class
def display_list(self):
    current_node = self.head
    elements = []
    while current_node:
        elements.append(str(current_node.data))
        current_node = current_node.next
    print("LinkedList: " + " -> ".join(elements) + " -> None")
```

Append (Add to End) To add a new node to the end, we first handle the case of an empty list. Otherwise, we traverse to the last node and update its **next** reference.

```

# Inside SinglyLinkedList class
def append(self, new_data):
    new_node = Node(new_data)

    # If the list is empty, the new node becomes the head
    if self.head is None:
        self.head = new_node
        return

    # Traverse to the last node
    last = self.head
    while last.next:
        last = last.next

    # Link the new node
    last.next = new_node

```

Insert at Beginning This is an efficient operation. The new node points to the current head, and then the new node becomes the head.

```

# Inside SinglyLinkedList class
def insert_at_beginning(self, new_data):
    new_node = Node(new_data)
    new_node.next = self.head # New node points to the old head
    self.head = new_node     # Update head to be the new node

```

Insert After a Given Value First, find the node containing the specified value, then insert the new node after it by rearranging the references.

```

# Inside SinglyLinkedList class
def insert_after_node(self, prev_node_data, new_data):
    current = self.head
    # 1. Find the node with prev_node_data
    while current:
        if current.data == prev_node_data:
            # 2. Insert the new node
            new_node = Node(new_data)
            new_node.next = current.next
            current.next = new_node
            return
        current = current.next

    # 3. If node was not found
    print(f"Node with data {prev_node_data} not found.")

```

Deletion To delete a node, we must find it while keeping track of the **previous** node, so we can "bypass" the node being deleted.

```

# Inside SinglyLinkedList class
def delete_node(self, key):
    current = self.head
    previous = None

    # Case 1: The head node holds the key
    if current and current.data == key:
        self.head = current.next
        return

    # Case 2: Search for the key, keeping track of the previous node
    while current and current.data != key:
        previous = current
        current = current.next

    # Case 3: The key was not present
    if current is None:
        return # Or print a message

    # Case 4: Unlink the node
    previous.next = current.next

```

Search A linear search iterates through the list until the key is found or the list ends.

```
# Inside SinglyLinkedList class
def search(self, key):
    current = self.head
    while current:
        if current.data == key:
            return True # Found
        current = current.next
    return False # Not found
```

2. Doubly Linked Lists (DLL)

A doubly linked list enhances an SLL by adding a **prev** reference to each node, allowing for traversal in both forward and backward directions.

The **DLLNode** Class

The node now contains three parts: data, a **next** reference, and a **prev** reference.

```
class DLLNode:
    def __init__(self, data):
        self.data = data
        self.prev = None # Reference to the previous node
        self.next = None # Reference to the next node
```

DLL Operations

Traversal (Forward and Backward) Forward traversal is identical to SLL. Backward traversal requires finding the last node and then following the **prev** references.

```

# Inside a DoublyLinkedList class
def traverse_forward(self):
    current = self.head # Assume 'head' is a DLLNode
    elements = []
    while current:
        elements.append(str(current.data))
        current = current.next
    print("Forward: None <-> " + " <-> ".join(elements) + " <-> None")

def traverse_backward(self):
    if self.head is None:
        return
    # Find the last node first
    last = self.head
    while last.next:
        last = last.next
    # Traverse back from the last node
    current = last
    elements = []
    while current:
        elements.append(str(current.data))
        current = current.prev
    print("Backward: None <-> " + " <-> ".join(elements) + " <-> None")

```

Append Appending requires updating two extra references: the new node's `prev` and the old last node's `next`.

```

# Inside a DoublyLinkedList class
def append(self, new_data):
    new_node = DLLNode(new_data)
    if self.head is None:
        self.head = new_node
        return
    last = self.head
    while last.next:
        last = last.next
    last.next = new_node # Link forward
    new_node.prev = last # Link backward

```

*Note: DLL insertion and deletion follow similar logic to SLL but require updating **prev** references as well, making the operations slightly more complex.*

3. Circular Singly Linked Lists (CSLL)

In a circular list, the **next** reference of the last node points back to the **head** node instead of **None**, creating a loop.

CSLL Operations

Traversal / Display Traversal requires a different stop condition to avoid an infinite loop. A common pattern is to loop until you return to the **head**.

```
# Inside a CircularSinglyLinkedList class
def display_csll(self):
    if self.head is None:
        print("CSLL is empty.")
        return

    current = self.head
    elements = []
    while True:
        elements.append(str(current.data))
        current = current.next
        if current == self.head: # Stop when we've circled back
            break
    print("CSLL: head -> " + " -> ".join(elements) + " -> (back to head)")
```

Append Appending to a circular list involves finding the last node (the one pointing to **head**) and inserting the new node, ensuring the new last node also points back to **head**.

```
# Inside a CircularSinglyLinkedList class
def append(self, new_data):
    new_node = Node(new_data)
    if self.head is None:
        self.head = new_node
        new_node.next = self.head # Points to itself
        return

    current = self.head
    while current.next != self.head:
        current = current.next

    new_node.next = self.head    # New node points to head
    current.next = new_node      # Old last node now points to new node
```

4. Applications Summary

- **Singly Linked Lists:** Ideal for memory-efficient, forward-only traversal. Used in implementing stacks, queues, and undo features.
- **Doubly Linked Lists:** Best when backward and forward traversal is needed. Used in browser history (back/forward), music playlists (next/previous), and LRU caches.
- **Circular Linked Lists:** Perfect for continuous loops. Used in round-robin scheduling for operating systems, multiplayer game turns, and repeating media playlists.

All materials on this website are for the exclusive use of teachers and students at subscribing schools for the period of their subscription. Any unauthorised copying or posting of materials on other websites is an infringement of our copyright and could result in your account being blocked and legal action being taken against you.

 **Report a problem**